1. Introduction

This document describes the work completed for the second implementation milestone of the Forensic Timeline Consistency and Evidence Constraint Analyzer. The application is being developed as a web-based system using Python, Flask, SQLite, HTML, and CSS. The pages use Flask and Jinja templates to display information. No custom JavaScript is used in the current implementation.

The Week 2 work focuses on the basic operations needed to record evidence events, define ordering relationships, run an initial consistency check, and show analysis results. This report separates functionality visible in the running application from features that are only partly implemented or still need testing.

2. Functional Requirements and Implementation

The original requirement identifiers are kept below. The status reflects the source code structure and the application behavior demonstrated so far. A feature is not treated as fully verified only because a file or page has been created.

ID	Requirement	Implementation completed	Responsible modules
FR-01	Add Evidence Events	Implemented in the application. The user can open the event form and submit a new event.	src/routes/events.py; src/services/event_service.py; templates/events/form.html
FR-02	Manage Evidence Events	Implemented. The event list provides view, edit, and delete operations.	src/routes/events.py; src/services/event_service.py; templates/events/list.html
FR-03	Enter Evidence Details	Implemented. The form accepts a description, start and end time, location, duration, and source type.	src/models/evidence_event.py; templates/events/form.html
FR-04	Define Event Relationships	Implemented in code. A user can define a BEFORE relationship between two saved events. The relationship count shown in the test screenshot was zero, so a successful relationship-based run is still to be demonstrated.	src/routes/constraints.py; src/services/constraint_service.py; src/database/schema.sql
FR-05	Construct Incident Timelines	Partly implemented. Events are ordered by their recorded start times and relationships are represented as a directed graph. A full	src/services/graph_service.py; src/services/event_service.py

		timeline display is not yet available.	
FR-06	Analyze Timeline Consistency	Initial implementation available. The analysis page ran and reported that two events were checked. The captured run checked zero ordering relationships, so this result does not verify BEFORE-rule checking.	src/algorithms/conflict_detection.py; src/algorithms/temporal_checks.py; src/routes/analysis.py
FR-07	Review Conflicts	Partly implemented. The results page has a section for conflicts and the analysis service stores conflict details. A screenshot showing a detected conflict has not yet been provided.	src/services/report_service.py; src/routes/analysis.py; templates/analysis/results.html
FR-08	Generate Alternative Timelines	Partly implemented. The application generates candidate event orders from the relationship graph for small acyclic cases. The displayed ordering needs a presentation correction and relationship-based behavior still needs testing.	src/algorithms/alternative_timelines.py; src/services/graph_service.py; templates/analysis/results.html
FR-09	Visualize Timeline Results	Not completed. The current page lists candidate orders but does not provide the planned full timeline visualization.	Future work: templates/analysis/ and timeline presentation component
FR-10	Import Evidence Data	Implemented in code for CSV and JSON files. A local import test should be run and its result recorded before claiming full verification.	src/routes/datasets.py; src/data_processing/csv_importer.py; src/data_processing/json_importer.py
FR-11	Export Evidence Data	Implemented in code. Evidence event records can be exported as CSV or JSON.	src/routes/datasets.py

FR-12	Test Analysis Scenarios	Partly implemented. Synthetic consistent and conflicting JSON examples are included. A user-facing scenario loader is not yet complete; files can be imported through the data import page.	datasets/synthetic/; src/data_processing/json_importer.py
FR-13	Analyze Public Dataset Records	Not completed. The NIBRS transformer is a placeholder. Dataset field mapping and conversion still need to be developed.	src/data_processing/nibrs_transformer.py
FR-14	Review Analysis Reports	Partly implemented. The report page shows the analysis status, summary, conflicts, and candidate event orders. More complete report details remain to be added.	src/services/report_service.py; templates/analysis/results.html
FR-15	Validate Evidence Inputs	Implemented in code. Required text fields, date/time values, time ordering, and non-negative numeric duration are checked before saving.	src/models/evidence_event.py; src/routes/events.py
FR-16	Preserve Original Evidence	Supported by the current design. Alternative event orders are generated separately and do not update the stored event records. Additional tests should confirm this behavior.	src/algorithms/alternative_timelines.py; src/database/schema.sql; src/services/event_service.py

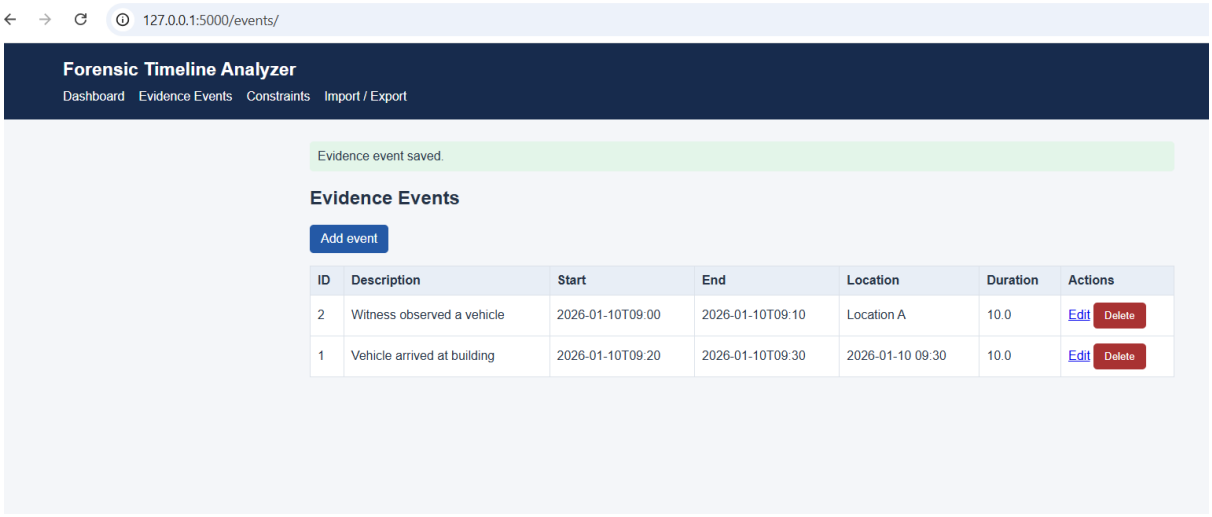
3. Main Components

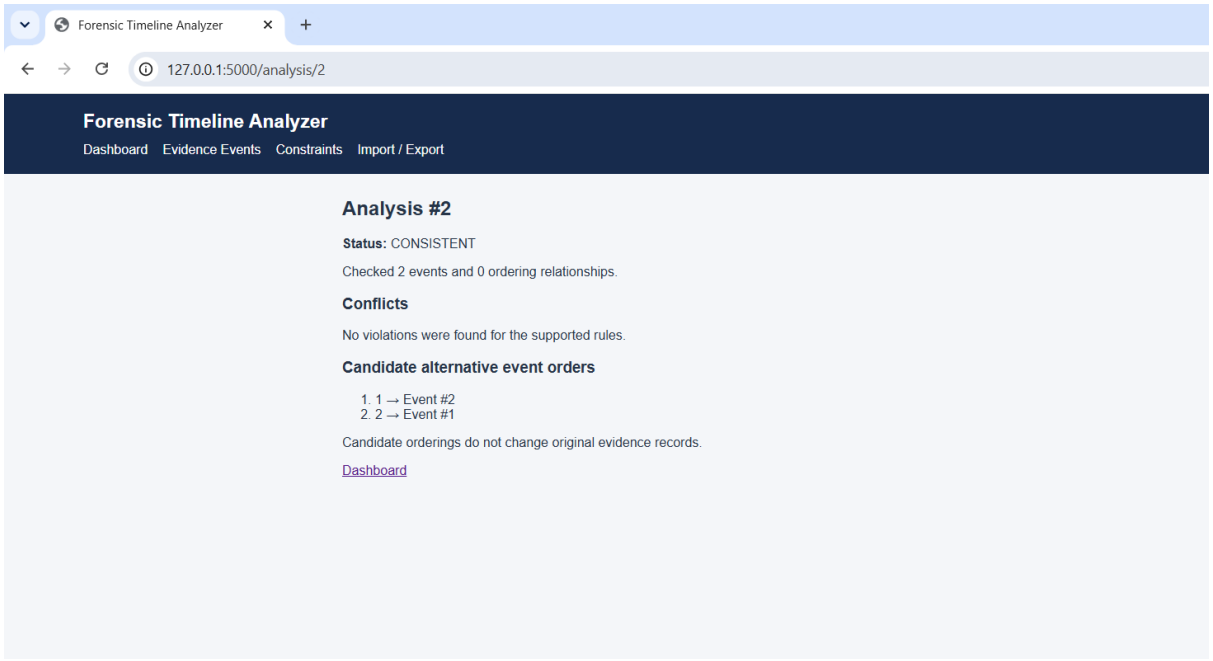
Component	Responsibility
app.py and src/__init__.py	Create and start the Flask application, set the template and static-file folders, initialize the database, and register routes.
src/routes/	Receive browser requests and connect the pages to application services.
src/models/evidence_event.py	Check event form values before they are saved.

src/services/event_service.py	Create, retrieve, update, and delete evidence event records in SQLite.
src/services/constraint_service.py	Save and retrieve BEFORE relationships between events.
src/services/graph_service.py	Build a directed graph from events and their ordering relationships using NetworkX.
src/algorithms/	Check time ordering, detect supported conflicts, and generate candidate topological orders.
src/services/report_service.py	Save analysis runs and their conflict details.
src/data_processing/	Read CSV and JSON files and provide the planned location for NIBRS transformation.
templates/ and static/css/	Provide the browser pages and their styling using HTML, Jinja, and CSS.
tests/	Contains automated tests for event operations, analysis, alternative orders, and data import/export.

4. Testing Performed and Results

The application was started locally in Visual Studio Code and opened in a web browser at http://127.0.0.1:5000/. The dashboard loaded after a template-folder configuration problem was corrected.





Test	Observed result
Application startup and dashboard	Passed based on the browser screenshot. The Flask page, navigation links, and dashboard content were displayed.
Template loading	Passed after updating src/__init__.py to point Flask to the project-level templates and static folders.
Analysis page rendering	Passed based on the supplied screenshot. The browser displayed Analysis #2, the status, summary, conflicts section, and candidate alternative orders.
Analysis run with two event records	The screenshot showed: 'Checked 2 events and 0 ordering relationships' and 'CONSISTENT'. This confirms the analysis route executed, but it does not prove that a BEFORE constraint was evaluated.
BEFORE relationship creation and checking	Not confirmed by the supplied analysis screenshot. The screenshot showed zero relationships. This test must be repeated after adding a BEFORE relationship.
Conflict detection with overlapping event times	Not yet confirmed by a supplied test result. Run a case where Event A ends after Event B starts, add A BEFORE B, and confirm that the report says INCONSISTENT.
Automated pytest suite	Not verified. Run pytest -q locally and record the actual result. Do not report the tests as passed until the command completes successfully.
CSV/JSON import and export	Implemented in source code; a complete browser test result has not yet been supplied.

5. Technical Problems Encountered

The first issue occurred when the application opened the dashboard. Flask raised jinja2.exceptions.TemplateNotFound: dashboard.html. The application factory was inside the src folder, while the templates folder was at the project root. Flask was therefore looking in the wrong location.

The problem was fixed by setting the template_folder and static_folder paths in src/__init__.py using the project root directory. After this change, the dashboard loaded successfully in the browser.

A second issue was noticed on the analysis results page. The alternative event order was displayed with unclear text, for example, '1 → Event #2'. The results template needs to display each item as 'Event #1 → Event #2'. This is a presentation issue and does not by itself show whether the generated ordering is correct.

The analysis screenshot also showed zero ordering relationships. The CONSISTENT result therefore only applies to the run shown and the checks actually performed. A separate test with a saved BEFORE relationship is required to demonstrate the constraint logic.

6. Incomplete Functionality and Further Work

- Add and verify a BEFORE relationship, then run analysis and confirm that the report counts the relationship.
- Run a conflicting scenario in which the first event ends after the second event starts. Confirm that the system reports INCONSISTENT and identifies the related events.
- Correct the alternative-order display and test that every generated order follows the saved BEFORE relationships.
- Complete a proper timeline visualization showing events and their relationships.
- Improve synthetic scenario handling so users can load the prepared consistent and conflicting scenarios directly and repeat the tests.
- Complete the NIBRS data mapping only after reviewing the fields available in the selected public dataset. Do not create missing timestamps or event details.
- Add fuller analysis reports, including affected event identifiers and explanations of each violated constraint.
- Run the complete automated test suite with pytest -q and record the actual output.
- Test CSV and JSON import/export through the browser and check how invalid rows are handled.
- Add tests to confirm that generated alternatives never change original evidence records.
- Machine-learning event classification is not part of the current Week 2 implementation. It should only be added if it supports a clear requirement in the approved proposal and suitable labelled data is available.

7. Week 2 Summary

The Week 2 work has moved the project beyond a basic Flask page. The application can start, display its dashboard, and open an analysis results page. The source code includes the main

parts for evidence-event management, SQLite storage, BEFORE relationships, initial consistency checks, conflict reporting, alternative ordering, and CSV/JSON processing.

The browser screenshots confirm that the dashboard and analysis page load. The analysis screenshot confirms that two event records were checked, but it also shows that no ordering relationships were included in that run. For this reason, the relationship and conflict tests must be completed before those functions are reported as fully tested. Timeline visualization and NIBRS processing remain future work.

8. How to Run and Test the Current Application

From the project root in the VS Code terminal, activate the virtual environment and install the dependencies:

```
py -m venv .venv

.venv\Scripts\Activate.ps1

pip install -r requirements.txt

python app.py
```

Open http://127.0.0.1:5000/ in a browser. To run the automated tests, use:

```
pytest -q
```

The test results should be copied into this report only after running the command on the development computer.